



សាកលវិទ្យាល័យភូមិន្ទភ្នំពេញ

Royal University of Phnom Penh

Implementation Documentation

Table of Contents

1. Overview	4
1.1. Purpose of the System.....	4
1.2. System Summary.....	4
1.3. System User & Role	4
1.4. Technologies Used.....	5
2. Microservices Architecture	5
2.1. Service Overview.....	5
2.2. Backend Service Detail	5
2.3. Communication.....	5
3. Project Directory.....	6
3.1. Backend	6
3.2. Frontend	7
4. Environment & Configuration.....	9
4.1. User Service – environment variable (.env)	9
4.2. Ads Service – environment variable(.env).....	9
4.3. Billing Service – environment variable (.env)	10
4.4. Kong-Gateway – environment variable (.env)	10
4.5. Web Socket – environment variable (.env)	10
4.6. Frontend – environment variable (.env)	11
4.7. Note	11
5. Setup & Installation	11
5.1. System requirement	11
5.2. Requirement Tool.....	11
5.3. Project Structure	12
5.4. Building the Service.....	12
5.5. Running all Service	12
5.6. Stop the Service	12
5.7. Updating code.....	12

6.	Module Implementation	12
6.1.	User Service	12
6.2.	Ads Service	13
6.3.	Billing Service.....	13
7.	Data Flow	14
7.1.	User Service	14
7.1.1.	User Registration Flow	14
7.1.2.	User Login Flow.....	14
7.2.	Ads Service Flow	15
7.2.1.	LED Flow.....	15
7.2.2.	Create Ads Flow	15
8.	Error Handling & Security.....	16
8.1.	Controller-Based Error handling.....	16
8.2.	Security.....	18
8.3.	Developer Reference / Implementation Notes.....	20
8.3.1.	Controller Usage Pattern.....	21
8.3.2.	Middleware Usage	22
8.3.3.	Security Notes.....	22
8.3.4.	Adding New Feature	22
8.3.5.	Repository / Code References.....	22
8.3.6.	API Gateway	22

1. Overview

System នេះត្រូវបានបង្កើតឡើងដោយ MERN Technology ដែលផ្ដោតទៅលើ Local Advertising System ដែលផ្ដោតទៅលើ Web។

1.1. Purpose of the System

គោលបំណងក្នុងការបង្កើត System នេះឡើងគឺចង់សម្រួលដល់ប្រព័ន្ធផ្សព្វផ្សាយសម្រាប់អាជីវកម្ម តូចធំសម្រាប់ Promote ផលិតផលរបស់ពួកគេក្នុងតម្លៃសមរម្យមួយទៅដល់ម្ចាស់អាជីវកម្ម ឬ ពាណិជ្ជករ។

1.2. System Summary

ដែលនៅក្នុងនេះយើងបានប្រើប្រាស់ Latest Technology ដែល Backend យើងបានប្រើប្រាស់ Node JS + Express JS , សម្រាប់ Database គឺផ្ដោតទៅលើ NoSQL (MongoDB) , Frontend យើងបានប្រើប្រាស់ Next JS ដែលជា Framework របស់ React JS និង Tailwind CSS ដែលជា Framework របស់ CSS សម្រាប់ការដាក់រូបសាយរបស់យើង ហើយ JWT សម្រាប់ Authentication.

1.3. System User & Role

- Admin - គ្រប់គ្រងរាល់ទិន្នន័យរបស់ System និង Approval Ads (Full control)
- Publisher / Advertiser - បង្កើតនិងគ្រប់គ្រង Ads របស់ខ្លួន

- Viewer / Anonymous User - អាចមើល Ads ដែលបាន Display នៅលើ

1.4. Technologies Used

- Backend: Node JS v24.11.1, Express JS v4.21.2, MongoDB v6.18.0, Mongoose v8.17.2
- Frontend: Next JS v15.5.0, Tailwind V4
- Dev Tools: VS Code, Insomnia
- Deployment: Vercel (frontend), Heroku/Hostinger (Backend)

2. Microservices Architecture

2.1. Service Overview

រាល់ Service នឹងដំណើរការដោយស្វ័យប្រវត្តន៍និងមានទំនាក់ទំនងគ្នាតាមរយៈ REST API ។

Database មានភាពដាច់ពីគ្នាដែល **1 database per service** ដោយមានភាពដាច់ពីគ្នា

2.2. Backend Service Detail

- User Service: រក្សាទុករាល់ទិន្នន័យដែលពាក់ព័ន្ធទៅនឹងអ្នកប្រើប្រាស់ គ្រប់គ្រង Authentication, JWT Generation, user CRUD
- Billing Service: រក្សាទុកឬកត់ត្រារាល់ទិន្នន័យដែលពាក់ព័ន្ធទៅនឹងការបង់ប្រាក់, ដកប្រាក់, សំណងជាដើម។
- Ads Service: រក្សាទុករាល់ទិន្នន័យ Ads របស់ Publisher/Advertiser ដែលបានដាក់ស្នើរ និង ផ្តល់ការបង្ហាញ Ads ព្រមទាំងទិន្នន័យដែលពាក់ព័ន្ធទៅនឹង LED។ Ads/LED CRUD
- Web Socket: សម្រាប់បង្ហាញ Ads real time
- Image Detector AI: សម្រាប់ប្រើប្រាស់ក្នុងការចាប់រាល់រូបភាពមិនសមរម្យនៅពេលដែល User upload
- Telegram Service: សម្រាប់ផ្តល់ Notification ដល់អ្នកប្រើប្រាស់

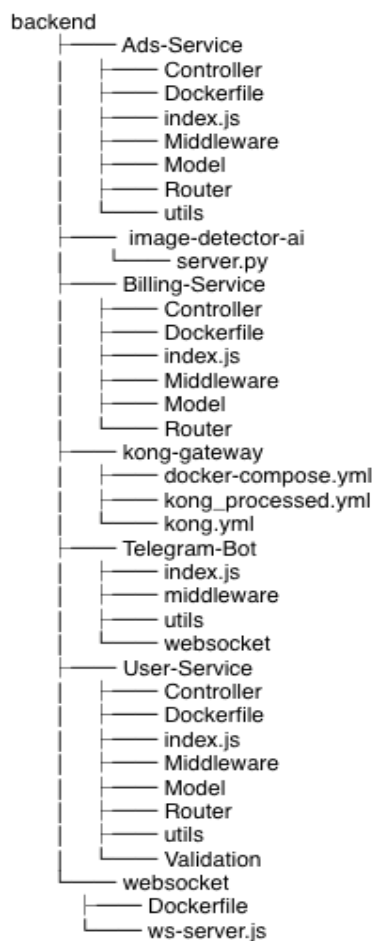
2.3. Communication

- រាល់ Service ទាំងអស់នឹងទំនាក់ទំនងគ្នាតាមរយៈ HTTP REST APIs

- Kong gateway ដើរតួយ៉ាងសំខាន់ក្នុងការទំនាក់ទំនងរវាងService នីមួយៗដែលជា API Gateway
 - Routes incoming request to correct microservice.
 - Handles authentication, rate limiting, and load balancing.

3. Project Directory

3.1. Backend

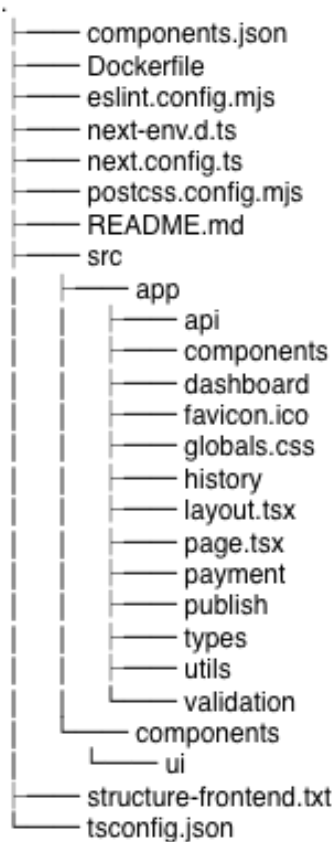


ដែលនៅខាងលើនេះគឺជា Project Directory របស់ Backend រាល់ Ads-Service , Billing-Service, User-Service គឺជាService នីមួយៗ

- File - index.js គឺជា main file ដែលសម្រាប់ Run service នីមួយៗដែលនៅក្នុងនោះ។

- Folder – Controller គឺជា Folder ដែលផ្ទុករាល់ការបញ្ជាប្រើ Code logic សម្រាប់បញ្ជា អោយធ្វើការអ្វីមួយជាមួយនឹង Database
- Folder – Middleware គឺជា Folder សម្រាប់ផ្ទុករាល់លក្ខខណ្ឌឬ Logic ដែលតម្រូវរាល់ការ Request ទាំងអស់ត្រូវឆ្លងកាត់ការពិនិត្យរបស់វាជាមុនសិន
- Folder – Model គឺជា Folder ដែល Define Schema របស់ Database។
- Folder – Router គឺជា Folder សម្រាប់បែកចែកទំនាក់ទំនងដែលអាចអោយតយើង Perform Controller ឬ ហៅ Controller ណាមួយមកប្រើតាមរយៈ Route នេះ
- Folder – Validation គឺជា Folder សម្រាប់ផ្ទុករាល់ទិន្នន័យសម្រាប់ឆែកការ Input, Request body ថាតើវាមានលក្ខណៈត្រឹមត្រូវដែរឬទេ
- Folder – Utils សម្រាប់ប្រើប្រាស់ជាជំនួយនៃ Program
- Folder – Image Detector AI ដែលនៅក្នុងនោះមាន server.py ដែលជា Image Detector ដែលយើងបាន Fine Tune
- File – Dockerfile ជា File សម្រាប់ Setup របស់ Docker ដែលងាយស្រួលសម្រាប់ Run Project

3.2. Frontend



- Folder - src/app ជា root project ដែលនៅក្នុង Next JS Structure
- Folder – API ដើរតួជា API នៅក្នុង Next JS សម្រាប់ Mask API Endpoint សម្រាប់ Client-Side Rendering
- Folder – app/components ដែលជា Folder សម្រាប់ Component ដែលយើងបង្កើតឡើងសម្រាប់ប្រើប្រាស់នៅក្នុង Page នីមួយៗ
- Folder – components ដែលនៅក្នុង Root directory ដែលជា Library component Shaden
- Folder – app/types ជាFolder សម្រាប់define type របស់ data
- Folder - ដែលរាល់ Folder ដែលមានជាឈ្មោះ dashboard, history etc ដែលនៅក្នុងនោះរាល់ Project ដែលមាន page.tsx គឺជា Folder landing page

4. Environment & Configuration

4.1. User Service – environment variable (.env)

MONGODB_URI	mongodb+srv://admin:yktkNbu853kdHI8v@globaladvertising.evf0ekn.mongodb.net/userDB?retryWrites=true&w=majority&appName=GlobalAdvertising
PORT	13001
SECRET_CREATE_WALLET_KEY	NotEasyAsYouExpectToBypassViaPostmanRequest
SECRET_WALLET_KEY	JustKeepGuessingTheWalletKey2.0
SECRET_KEY	GlobalAdvertisingSecretKey2.0
AWS_ACCESS_KEY	AKIAWCYP3662LUREFN76
AWS_SECRET_KEY	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
AWS_REGION	ap-southeast-1
AWS_BUCKET_NAME	globaladvertisingstorage
FRONTEND_URL	http://localhost:13000
BACKEND_URL	http://user:13001/
EMAIL_USER	chhievton@gmail.com
EMAIL_PASS	xdkm uomw vswm ofii
ADS_SERVICE_URL	http://ads:13002
BILLING_SERVICE_URL	http://billing:13003
BOT_USERNAME	lct_local_advertising_bot
BOT_TOKEN	7423506344:AAERjuBAGEg8Tcss2Niy9PmoDKalDI1p7o

4.2. Ads Service – environment variable(.env)

MONGODB_URI	mongodb+srv://admin:yktkNbu853kdHI8v@globaladvertising.evf0ekn.mongodb.net/adsDB?retryWrites=true&w=majority&appName=GlobalAdvertising
PORT	13002
FRONTEND_URL	http://frontend:13000
SECRET_KEY	GlobalAdvertisingSecretKey2.0
SECRET_WALLET_KEY	JustKeepGuessingTheWalletKey2.0
SECRET_ADS_KEY	AnotherSecretKeyForAds2.0

AWS_ACCESS_KEY	AKIAWCYP3662LUREFN76
AWS_SECRET_KEY	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
AWS_REGION	ap-southeast-2
AWS_BUCKET_NAME	globaladvertisingstorage
BOT_TOKEN	7423506344:AAERjuBAGEg8Tcss2Niy9PmoDKalDl1p7o
BILLING_SERVICE_URL	http://billing:13003

4.3. Billing Service – environment variable (.env)

MONGODB_URI	mongodb+srv://admin:ytkNbu853kdHI8v@globaladvertising.evf0ekn.mongodb.net/billingDB?retryWrites=true&w=majority&appName=GlobalAdvertising
PORT	13003
SECRET_KEY	GlobalAdvertisingSecretKey2.0
SECRET_CREATE_WALLET_KEY	NotEasyAsYouExpectToBypassViaPostmanRequest
SECRET_WALLET_KEY	JustKeepGuessingTheWalletKey2.0

4.4. Kong-Gateway – environment variable (.env)

KONG_USERS_SERVICE_URL	http://host.docker.internal:3001
KONG_ADS_SERVICE_URL	http://host.docker.internal:3002
KONG_LED_SERVICE_URL	http://host.docker.internal:3002
KONG_BILLINGS_SERVICE_URL	http://host.docker.internal:3003

4.5. Web Socket – environment variable (.env)

ADS_SERVICE_URL	http://localhost:13002
ADMIN_TOKEN	
SECRET_ADS_KEY	AnotherSecretKeyForAds2.0

4.6. Frontend – environment variable (.env)

API_GATEWAY_URL	http://kong:8000
SECRET_KEY	GlobalAdvertisingSecretKey2.0
STRIPE_WEBHOOK_SECRET	whsec_54ca8299055670f7f aabac42bada993beb03 58f6ea96060f78368f6fb1210caf
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	pk_test_51RzfCTGb0CqGweJaBV0 ihJO5UUiagD4V65kxcu7FIziDa EWb6FIGar5xztqOv8RthDvj EdoulplkleSmUPaiQAph006tqUVndh
STRIPE_SECRET_KEY	sk_test_51RzfCTGb0CqGweJa XrNmCOQ02pcmqeJUyV7yLa XsNIRKwjEPk416BpP5cv2c2AK5x7UjlqORc9YFgZqo7NDuLLKR00 vHLFVzAw
SECRET_WALLET_KEY	JustKeepGuessingTheWalletKey2.0
SECRET_ADS_KEY	AnotherSecretKeyForAds2.0
HF_API_KEY	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
NEXT_PUBLIC_WS_URL	ws://websocket-service:13010

4.7. Note

- Each microservice manages its own DB and Config
- Environment Variable must be set before running the service

5. Setup & Installation

5.1. System requirement

- Operating System: macOS / Windows / Linux
- Minimum RAM: 8GB
- Disk Space: 2–5GB for Docker images

5.2. Requirement Tool

- Docker (version 24+ recommend)
- Docker Compose (version 2.0+)

5.3. Project Structure

Local-Advertising

- Backend
 - Ads-Service
 - Billing-Service
 - User-Service
 - websocket
 - Telegram-Bot
 - kong-gateway
 - image-detector-ai
- Frontend
- docker-compose.yml

5.4. Building the Service

- docker compose run build – build all service.
- docker compose build user – build specific service.

5.5. Running all Service

- docker compose up.

5.6. Stop the Service

- docker compose down.

5.7. Updating code

- docker compose build <service_name>
- docker compose up.

6. Module Implementation

6.1. User Service

- Controller

- userController – CRUD Operation for User, handling login, register, forgot password, verify account, JWT Creation etc
- Router
 - userRouter – map endpoint to controller (/login , /register, /verify-email, forgot-password, change password, /update-profile etc.
- Middleware
 - verifyToken – JWT Checking, Verify user identity.
- Validation
 - userValidation – Validation user input

6.2. Ads Service

- Controller
 - AdsController – CRUD Operation for Ads, upload to AWS Storage, time booking etc
 - LEDController – CRUD Operation for LED Display
 - telegramBotController – Performing notification for user and admin
- Router
 - AdsRouter – map endpoint to controller (/all, /pending, /book-ranges, /create etc)
 - LEDRouter – CRUD Operation for LED
- Middleware
 - verifyToken – JWT Checking, Verify user identity.
 - upload – Checking file upload format and sizes.

6.3. Billing Service

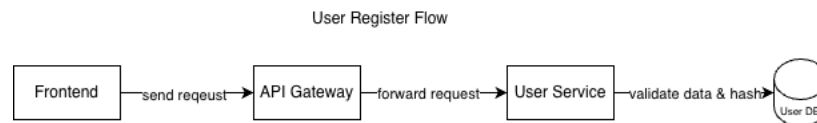
- Controller
 - walletController – CRUD Operation (get , create , deposit , deduct wallet)
 - adsTransactionController – Create ads transaction and Get transaction detail.

7. Data Flow

7.1. User Service

7.1.1. User Registration Flow

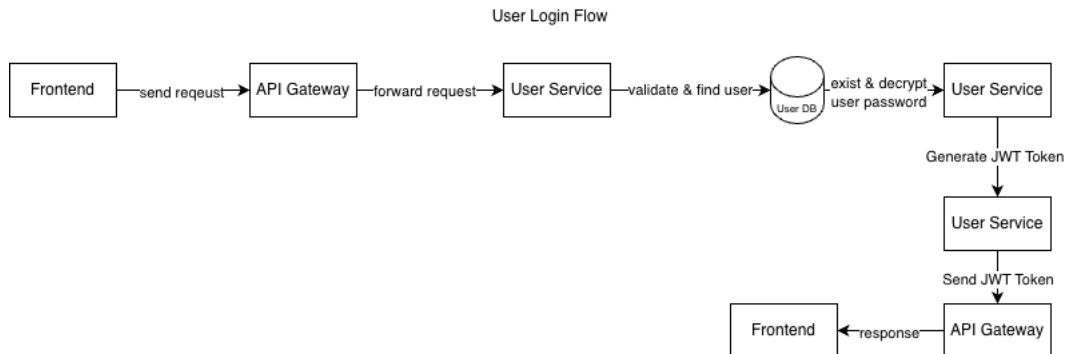
- Frontend send request to API Gateway for register account.
- API Gateway received request check rate limit and then find service that match User Service API
- User Service validates Data
- User Service hashed password and save to User Service Database.



7.1.2. User Login Flow

- Frontend send request to API Gateway for Login into account.
- API Gateway receives the request and performs:
 - Rate limiting check
 - Routing to the appropriate microservice (User Service)
- API Gateway forwards the request to the User Service.
- User Service validates the incoming login data (email, password).
- User Service searches the database for the user account.
- If user exists, User Service compares the password (hashed).
- If valid, User Service generates a JWT token.
- User Service sends the JWT token response back to API Gateway.

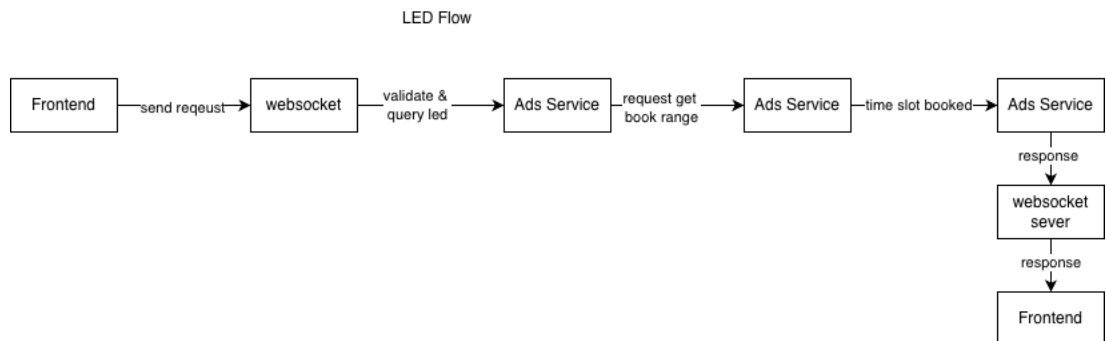
- API Gateway forwards the response to the Frontend.



7.2. Ads Service Flow

7.2.1. LED Flow

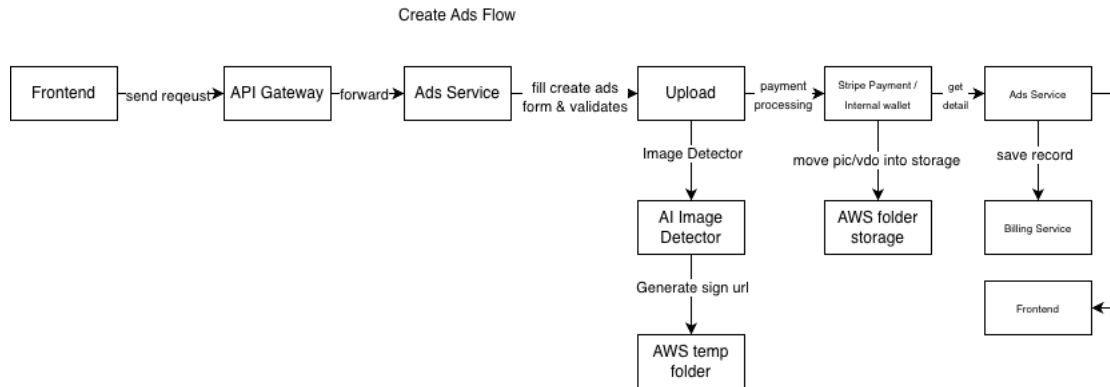
- Frontend send request to websocket for checking the book range or available time.
- Websocket return the available time for ads to display
- Frontend received the time and block the time that publisher create ads already
- Frontend allow user to choose and select the time available to make an advertising



7.2.2. Create Ads Flow

- Frontend send request API Gateway
- API Gateway receives the request and performs:
 - Rate limiting check
 - Routing to the appropriate microservice (Ads Service)
- User Upload Image/Video

- AI Image Detect sensitive (validates image)
- Generate sign URL to AWS Temporary Folder
- Payment processing
- Move Image/Video from temporary folder to AWS Folder
- Save record to Billing Service
- Response to Frontend



8. Error Handling & Security

8.1. Controller-Based Error handling

- Each controller use try/catch to handle errors.
- Standard response format with HTTP Status Code:


```

{
  "success" : false,
  "message" : "Error .....",
}
      
```
- Common status codes:
 - 400 – Validation Error
 - 401 – Authorization Error
 - 403 – Forbidden Error / Middleware Error
 - 404 – Not Found
 - 429 – Too many request / Rate limit
 - 500 – Internal Server Error

- Example : Simplified Login Controller

```
Login Controller

const login = asyncHandler(async (req, res) => {
  const { email, password } = req.body;

  // 1. Validate input
  if (!email || !password) {
    return res.status(400).json({ success: false, message: "Email and password required" });
  }

  // 2. Find user
  const user = await User.findOne({ email });
  if (!user || !user.password) {
    return res.status(401).json({ success: false, message: "Invalid credentials" });
  }

  // 3. Compare password
  const match = await bcrypt.compare(password, user.password);
  if (!match) {
    return res.status(401).json({ success: false, message: "Invalid credentials" });
  }

  // 4. Generate JWT token
  const token = jwt.sign(
    { id: user._id, role: user.role },
    process.env.SECRET_KEY,
    { expiresIn: "7d" }
  );

  res.status(200).json({ success: true, token, message: "Logged in successfully" });
});
```

- Example getUserProfile Controller

```

getUserProfile Controller

const getUserProfile = asyncHandler(async (req, res) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ success: false, message: "Unauthorized" });
  }
  const token = authHeader.split(" ")[1];

  try {
    const decoded = jwt.verify(token, process.env.SECRET_KEY);
    const user = await User.findById(decoded.id).select("-password");

    if (!user) {
      return res.status(404).json({ success: false, message: "User not found" });
    }

    res.status(200).json({ success: true, data: user });
  } catch (error) {
    res.status(401).json({ success: false, message: "Invalid token" });
  }
});

```

8.2. Security

- JWT authentication for all protected routes
- Role-based access checks implemented in middleware.
- Input validation in controllers using zod validation.
- Each service when communication with each other it has internal key for security.
- Example: Token Validation

```
verifyToken Middleware

// Verify JWT token for protected routes
const verifyToken = async (req, res, next) => {
  let token = req.header("Authorization");
  if (!token) {
    return res.status(401).json({ error: "Access Denied!" });
  }

  token = token.replace("Bearer ", "");

  try {
    const decode = jwt.verify(token, process.env.SECRET_KEY);
    const user = await User.findById(decode.id).select(
      "id email role telegram.chatId telegram.isConnected"
    );

    if (!user) {
      return res.status(404).json({ error: "User not found" });
    }

    req.user = user;
    next();
  } catch (error) {
    res.status(400).json({ error: "Invalid Token" });
  }
};
```

- Example: Admin Validation

```
verifyAdmin Middleware

// Only allow admin users
const verifyAdmin = (req, res, next) => {
  if (req.user.role !== "admin") {
    return res.status(403).send("Access denied");
  }
  next();
};
```

- Example: Allowed Publisher or Admin

```
verifyPublisher Middleware

// Allow publisher or admin users
const verifyPublisher = (req, res, next) => {
  if (req.user.role === "publisher" || req.user.role === "admin") {
    return next();
  }
  return res.status(403).send("Access denied");
};
```

8.3. Developer Reference / Implementation Notes

Provide concise guidance for developers to understand the implementation, use controllers/middleware, and follow patterns consistently. More detail about API route checks it out in **Design Documentation**

Endpoint	Method	Auth Required	Success Response	Error Response
/api/users/login	POST	No	200 -> JWT Token	400 – Email and password are required 401 - Invalid email or password
/api/users/register	POST	No	200 -> User Created	400 -> Missing field 409 -> Email Already exist
/api/users/me	GET	JWT	200 -> User Data	400 -> Invalid Token
/api/ads/create	POST	JWT + Publisher/Admin	201 -> Ads Created	401 -> Unauthorized 403 → Forbidden
/api/leds/all	GET	No	200 -> LED Data	No
/api/wallets/me	GET	JWT	200 -> Wallet data	401 -> Unauthorized
/api/wallets/create	POST	JWT + Internal Key	200 -> Deposit Created	403 -> Forbidden 409 -> Wallet already exist

8.3.1. Controller Usage Pattern

- All controller use try/catch to handle error
- Response standard:

```

{
  "message": "Sucessfully ... ",
  "data": {...} or [{...}]
}

```

- Asynchronous calls in controllers should use `asyncHandler` (or equivalent) to catch unhandled promise errors.

8.3.2. Middleware Usage

- `verifyToken` – Use for verifying or checking the token is it valid or not and allowed to process to the next task after token is valid
- `verifyAdmin` – Allowed only admin to access this middleware
- `verifyPublisher` – Publisher & Admin routes
- `tryVerifyToken` - optional authentication for public routes

8.3.3. Security Notes

- Any service to service directly we need internal key for security when someone trying to call API via curl, postman etc
- Input validation should be done using ZOD.
- Always use HTTPS for API requests.

8.3.4. Adding New Feature

- Follow the same patterns for consistency.
 - Use standardized for JSON response.
 - Apply middleware in the correct order.
 - Add try/catch in async controller.
 - Log errors appropriately if needs.
 - Frontend side make sure create Next API for masked API Endpoint when called in Client Side Rendering(CSR).

8.3.5. Repository / Code References

- Where to find the code
 - Controller: `/backend/<service_name>/Controller`
 - Middleware: `/backend/<service_name>/Middleware`
 - Router: `/backend/<service_name>/Routers`
 - Model: `/backend/<service_name>/Model`
 - Validation: `/backend/<service_name>/Validation`

8.3.6. API Gateway

- Frontend most called through API Gateway do not direct into Each Service

- Route Delegation
 - /api/users/* - User Service
 - /api/ads/* - Ads Service
 - /api/led/* - Ads Service
 - /api/billing/* - Billing Service
 - /api/wallets/* - Billing Service